Solutions for Tutorial problems on LPC2148 hardware interfacing

1)

Literature Survey :

Electromechanical relays : let low-power control circuits switch higher voltages and currents by energizing a coil that mechanically closes or opens contacts (typically COM/NO/NC). Because a coil draws tens to hundreds of milliamps and produces a large inductive voltage when de-energized, a microcontroller's GPIO cannot drive it directly; a transistor stage and a flyback path are required to both supply current and suppress the back-EMF. Integrated driver arrays such as the **ULN2803A** are commonly used in this role: each channel is a Darlington NPN transistor with a built-in base resistor and a clamp diode to a common pin (**COM**). When the **COM** pin is tied to the relay-supply rail (e.g., +12 V), the internal diode safely returns the coil's inductive kick, eliminating the need for an external diode across the coil.

ULN2803A: accepts standard TTL/CMOS logic levels and behaves as an inverting low-side switch: a logic HIGH on an input turns on the corresponding transistor so the output pin **sinks** current to ground, energizing the coil whose other end is tied to the positive supply. Each channel can sink up to approximately 500 mA, which comfortably covers most small signal relays.

LPC2148 : provides two 32-bit GPIO ports (P0 and P1). After reset, pins default to inputs and their functions are selected via **PINSEL** registers; the **IODIR** register configures direction; outputs are driven atomically using **IOSET** (to write 1s) and **IOCLR** (to write 0s), and read via **IOPIN**. In this design, **P0.10** is configured as a GPIO output and connected to **ULN2803A IN1**. The corresponding **OUT1** pin of the driver sinks the coil current, while **COM** is tied to the relay supply (e.g., +12 V) to engage the internal clamp diode. With this arrangement, driving P0.10 HIGH energizes the relay; driving it LOW de-energizes it, allowing simple software control with reliable electrical protection.

Table 64. GPIO pin description

Pin	Type	Description
P0.0-P.31 P1.16-P1.31	Input/ Output	General purpose input/output. The number of GPIOs actually available depends on the use of alternate functions.

C Program

Assumptions (typical LPC2148 board): 12 MHz crystal, PLL → **CCLK = 60 MHz**; set **VPBDIV=1** so **PCLK = 60 MHz**. If your clock differs, adjust **T0PR**.

```
#include <LPC214x.h>

#define RELAY_BIT   10u                 // P0.10

#define RELAY_MASK  (1u << RELAY_BIT)


static void gpio_init(void) {

    /* Make P0.10 a GPIO pin: PINSEL0 bits [21:20] = 00 */

    PINSEL0 &= ~(0x3u << (RELAY_BIT * 2));

    /* Set P0.10 as output */

    IODIR0  |= RELAY_MASK;

}


static void timer0_init_1ms(void) {

    VPBDIV = 0x01;                  // PCLK = CCLK for neat math

    T0TCR  = 0x02;                  // reset Timer0

    T0PR   = (60000000u/1000u) - 1u; // prescale so TC ticks every 1 ms

    T0TCR  = 0x01;                  // enable Timer0

}


static void delay_ms(unsigned ms) {

    unsigned start = T0TC;

    while ((T0TC - start) < ms) { /* spin */ }

}
```

```
int main(void) {
    gpio_init();
    timer0_init_1ms();

    while (1) {
        IOSET0 = RELAY_MASK;   // Relay ON (ULN2803A inverts)
        delay_ms(50);
        IOCLR0 = RELAY_MASK;   // Relay OFF
        delay_ms(50);
    }
}
```

2)

Literature Survey

The LPC2148 microcontroller contains a 10-bit Digital to Analog Converter (DAC) with a resistor string architecture. The DAC output is available on pin **P0.25 (AOUT)**, which is enabled by configuring **PINSEL1 bits 19:18 = 10**. The output voltage of the DAC is determined by the DACR register, where bits [15:6] hold the 10-bit digital value and bit 16 (BIAS) controls the speed versus power trade-off. With BIAS = 0, the settling time is 1 µs and maximum current is 700 µA. The analog output voltage is given by:

$$V_{AOUT} = \frac{\text{VALUE}}{1024} \times V_{REF}$$

Ref : Table 286. DAC pin description, UM10139 LPC214x User manual

Pin	Type	Description
AOUT	Output	**Analog Output.** After the selected settling time after the DACR is written with a new value, the voltage on this pin (with respect to V_{SSA}) is VALUE/1024 * V_{REF}.
V_{REF}	Reference	**Voltage Reference.** This pin provides a voltage reference level for the D/A converter.
V_{DDA}, V_{SSA}	Power	**Analog Power and Ground.** These should be nominally the same voltages as V_3 and V_{SSD}, but should be isolated to minimize noise and error.

DAC Calculation:

In this problem, the DAC must generate a periodic waveform that rises linearly from 0 V to VREF, holds at VREF for 100 ms, and then drops instantly to 0 V. The exact duration of the rising ramp is not specified in the problem statement. For calculation purposes, we **assume that the ramp duration is also 100 ms**, so that the total period of the waveform is

$$T = T_{\text{ramp}} + T_{\text{hold}} = 100\,\text{ms} + 100\,\text{ms} = 200\,\text{ms}.$$

This corresponds to a waveform frequency of

$$f = \frac{1}{T} = \frac{1}{0.2} = 5\,\text{Hz}.$$

Using the 10-bit resolution of the DAC, there are 1024 steps in the ramp. Hence, each increment must occur at intervals of

$$\Delta t = \frac{T_{\text{ramp}}}{1024} = \frac{100\,\text{ms}}{1024} \approx 97.6\,\mu\text{s}.$$

The piecewise function describing one cycle of the waveform is therefore:

$$v(t) = \begin{cases} \frac{V_{REF}}{T_{\text{ramp}}} \cdot t, & 0 \le t < T_{\text{ramp}} \\ V_{REF}, & T_{\text{ramp}} \le t < T \\ 0, & t = T \end{cases} \quad \text{with periodic extension } v(t+T) = v(t).$$

C Program:

To generate the required waveform, we configure the on-chip DAC of LPC2148.

1. The DAC output is available on pin P0.25 (AOUT), which is enabled by setting PINSEL1[19:18] = 10.

2. The DACR register is used to control the DAC. Bits [15:6] carry the 10-bit VALUE (0–1023), and bit 16 (BIAS) selects settling time (BIAS = 0 → 1 µs, fast mode).

3. To create the waveform:

    - Write values 0 → 1023 step by step with ~98 µs delay between steps. This gives a linear ramp of 100 ms.
    - Hold VALUE = 1023 for 100 ms to maintain the output at VREF
    - Finally, write VALUE = 0 for an instantaneous drop.
    - Repeat this cycle continuously, giving a total period of 200 ms (5 Hz).

```
#include <LPC214x.h>
#include <stdint.h>

/* Delay functions (approximate, adjust as per PCLK) */
void delay_us(unsigned int us) {
    volatile unsigned int i, j;
    for (i = 0; i < us; i++) {
        for (j = 0; j < 15; j++);   // ~1 µs delay
    }
}

void delay_ms(unsigned int ms) {
    while (ms--) delay_us(1000);
}

/* DAC Initialization */
void DAC_init(void) {
    PINSEL1 &= ~(3u << 18);   // Clear bits 19:18
    PINSEL1 |=  (2u << 18);   // Select AOUT (DAC output) on P0.25
}

/* Write 10-bit value to DAC */
void DAC_write(uint16_t value) {
    DACR = ((value & 0x03FF) << 6) | (0 << 16);   // VALUE→15:6, BIAS=0
}

int main(void) {
    uint16_t i;
    DAC_init();

    while (1) {
        /* Ramp up: 0 → 1023 in 100 ms */
        for (i = 0; i <= 1023; i++) {
            DAC_write(i);
```

```
            delay_us(98);   // ~97.6 µs per step
        }

        /* Hold at VREF for 100 ms */
        DAC_write(1023);
        delay_ms(100);

        /* Drop to 0 instantly */
        DAC_write(0);
    }
}
```

3)

To establish serial communication between the LPC2148 microcontroller and an external device (such as a PC terminal), the Universal Asynchronous Receiver/Transmitter (UART0) is configured. In this program, UART0 is initialized on pins P0.0 (TxD0) and P0.1 (RxD0) by setting the appropriate values in the `PINSEL0` register. The communication parameters are fixed at 8 data bits, no parity, and 1 stop bit (8-N-1) as per the requirement.

Table 167: Baudrates available when using 20 MHz peripheral clock (PCLK = 20 MHz)

Desired baudrate	MULVAL = 0 DIVADDVAL = 0			Optimal MULVAL & DIVADDVAL		
	U0DLM:U0DLL hex[2]	U0DLM:U0DLL dec[1]	% error[3]	U0DLM:U0DLL dec[1]	Fractional pre-scaler value MULDIV / MULDIV + DIVADDVAL	% error[3]
3600	015B	347	0.0640	248	5/(5+2)	0.0064
4800	0104	260	0.1600	125	12/(12+13)	0.0000
7200	00AE	174	0.2240	124	5/(5+2)	0.0064
9600	0082	130	0.1600	93	5/(5+2)	0.0064
19200	0041	65	0.1600	31	10/(10+11)	0.0064
38400	0021	33	1.3760	12	7/(7+12)	0.0594
56000	0021	22	1.4400	13	7/(7+5)	0.0160
57600	0016	22	1.3760	19	7/(7+1)	0.0594
112000	000B	11	1.4400	6	7/(7+6)	0.1600
115200	000B	11	1.3760	4	7/(7+12)	0.0594
224000	0006	6	7.5200	3	7/(7+6)	0.1600
448000	0003	3	7.5200	2	5/(5+2)	0.3520

Using the **LPC214x User Manual (Table 167: Baudrates for PCLK = 20 MHz)**, the divisor values for 3600 bps are selected. The optimal configuration corresponds to:

- `U0DLL = 248`, `U0DLM = 0`
- `MULVAL = 5`, `DIVADDVAL = 2`

This yields a baud rate of approximately **3600 bps with only 0.0064% error**, which is acceptable. The following C program implements this setup and transmits the string **"Hello World"** via UART0.

```
#include <LPC214x.h>
#include <stdint.h>

/* UART0 Initialization */
void UART0_init(void)
{
   // Configure P0.0 (TxD0) and P0.1 (RxD0) for UART0 function
   PINSEL0 &= ~((3u << 0) | (3u << 2));   // Clear function bits
   PINSEL0 |=  (1u << 0) | (1u << 2);     // Set to 01 for TxD0, RxD0

   // Enable DLAB = 1, select 8-bit word length, 1 stop bit, no parity
   U0LCR = (1 << 7) | 0x03;

   // Baud rate setting: 9600 bps @ PCLK = 15 MHz → DLL = 97 (0x61), DLM = 0
   U0DLL = 0x61;     // LSB of divisor
   U0DLM = 0x00;     // MSB of divisor

   // Clear DLAB after setting baud rate, lock configuration
   U0LCR &= ~(1 << 7);
}

/* Function to receive a byte from UART0 */
unsigned char UART0_RxChar(void)
{
   while ((U0LSR & (1 << 0)) == 0);   // Wait until RDR = 1 (data available)
   return U0RBR;                      // Return received byte
}

/* Function to transmit a byte via UART0 */
void UART0_TxChar(char ch)
{
   while ((U0LSR & (1 << 5)) == 0);   // Wait until THRE = 1 (THR empty)
   U0THR = ch;                        // Load character into THR
}

/* Function to transmit a string */
void UART0_SendString(char *str)
{
   while (*str != '\0')
   {
      UART0_TxChar(*str++);
   }
}
```

```
/* Main Program */
int main(void)
{
   char ch;
   UART0_init();                          // Initialize UART0

   UART0_SendString("Hello World\r\n");        // Display message once

   while (1)
   {
      ch = UART0_RxChar();                  // Receive a character
      UART0_SendString("Received : ");
      UART0_TxChar(ch);                     // Echo it back
      UART0_SendString("\r\n");
   }
}
```